

Jurnal Praktikum Dasar Kecerdasan Artifisial

Modul 7: Depth First Search (DFS)

Tujuan Praktikum

- Mahasiswa memahami dan mampu mengimplementasikan algoritma *Depth-First Search* pada Python dengan menggunakan *library* NetworkX.

Petunjuk Umum

1. Jangan lupa untuk selalu import *library* NetworkX & Matplotlib ketika memulai sesi baru (run time) atau kode Python yang memerlukan penggunaan *library* NetworkX
2. Jangan ubah struktur kode yang diberikan, cukup isi bagian yang kosong.
3. Jalankan setiap sel satu per satu agar tidak ada error.
4. Kerjakan soal yang ada secara berurutan, dikarenakan soal akan meneruskan code soal sebelumnya

```
In [ ]: print("") # Tuliskan nama Lengkap Anda
        print("") # Tuliskan NIM Anda
        print("IF-48-XX") # Tuliskan kelas Anda
        print("") # Tuliskan kode asprak anda
        print("") # Tuliskan nomor meja Anda
```

```
In [1]: import networkx as nx # Library untuk membuat graf
        import matplotlib.pyplot as plt # Library untuk plot grafik
```

Fungsi pendukung untuk mencetak graf

!! Tidak usah dimodifikasi !!

```
In [2]: # Posisi node
        pos = {
            "Home": (0, 0),
            "Projects": (-1.5, -1),
            "Music": (0, -1),
            "Downloads": (1.5, -1),
            "App1.py": (-2, -2),
            "Report.pdf": (-1, -2),
            "Song1.mp3": (0, -2),
            "Installer.exe": (1.5, -2)
        }
```

```
In [3]: # Fungsi pendukung untuk mencetak graf
        def show_graph(G, pos=None, title='') :
            # Membuat pos jika pos tidak diberikan
```

```

if pos is None:
    pos = nx.spring_layout(G)

# Fungsi untuk menggambar node
nx.draw(
    G,                    # Graf NetworkX
    pos,                  # Posisi node
    with_labels=True,     # Menampilkan nama node
    node_color='green',   # Warna node
    node_size=2000,       # Ukuran node
    font_color="white",    # Warna font Label node
    font_weight="bold",    # Berat font Label node
    font_size=7,          # Ukuran font Label node
    width=5               # Ketebalan garis edge
)

# Mengambil Label edge jika ada weight
edge_labels = nx.get_edge_attributes(G, 'weight')
# Fungsi untuk menggambar node
nx.draw_networkx_edge_labels(
    G,
    pos,
    edge_labels=edge_labels, # Data weight
    font_color='purple',    # Warna font Label edge
    font_weight="bold",     # Berat font Label edge
    font_size=16,           # Ukuran font Label edge
)

plt.margins(0.2) # Memberikan margin pada plot
plt.title(title) # Menampilkan judul graf jika diberikan
plt.show()      # Menampilkan graf menggunakan matplotlib

```

1. Implementasi DFS (Arsitektur Dasar) (50)

Rina adalah seorang programmer yang memiliki banyak proyek dan file di komputernya. Sebagai seorang programmer yang aktif mengerjakan berbagai proyek freelance dan tugas kantor, struktur folder di komputer Rina menjadi sangat kompleks dan bercabang-cabang, dengan banyak sub-folder dan file yang tersimpan di berbagai lokasi.

Anda diminta untuk membuat sebuah program yang dapat membantu Rina melakukan penelusuran direktori menggunakan algoritma DFS. Program ini diharapkan dapat mempermudah Rina dalam menemukan file yang ia butuhkan.

a. (5 poin) Inisialisasi graf bernama `data_rina` menggunakan `graf berarah` untuk menunjukkan arah perjalanan.

In [4]:

```
# Inisialisasi graf berarah
data_rina = nx.DiGraph()
```

b. (10 poin) Tambahkan node ke dalam graf yang merepresentasikan folder dan file. Folder tersebut terdiri dari:

- Home
- Projects

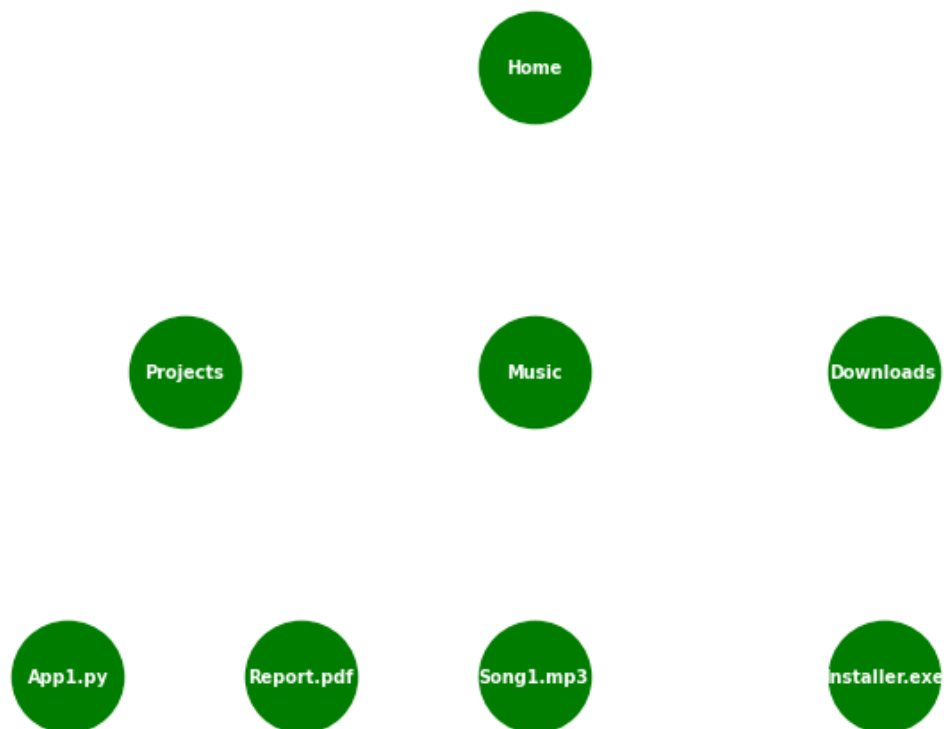
- Music
- Downloads
- App1.py
- Report.pdf
- Song1.mp3
- Installer.exe

Pastikan semua node tersebut ditambahkan ke dalam graf sehingga setiap bagian dapat direpresentasikan dengan baik dan nantinya dapat dihubungkan oleh edge untuk menunjukkan rute di antara bagian-bagian tersebut.

```
In [5]: # List daftar nama file dan folder yang terdapat pada komputer Rina
nodes = ["Home", "Projects", "Music", "Downloads", "App1.py", "Report.pdf", "Song1.mp3", "Installer.exe"]

# Tambahkan node dari list variabel `nodes` pada graf data_rina
data_rina.add_nodes_from(nodes)

# Tampilkan graf data_rina setelah penambahan node
show_graph(data_rina, pos=pos)
```



Contoh output:

 Contoh output penambahan lokasi

c. (10 poin) Berikut adalah daftar isi dari setiap folder. Hubungkan tree berdasarkan isi dari setiap folder pada komputer Rina.

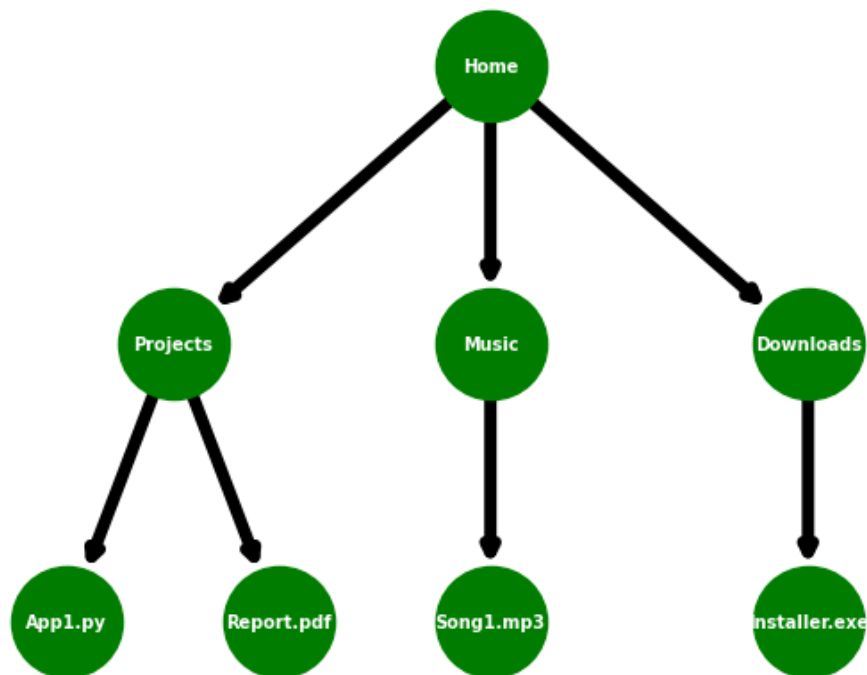
- Home berisi Projects, Music, dan Downloads.
- Projects berisi App1.py dan Report.pdf.

- Music berisi Song1.mp3 .
- Downloads berisi Installer.exe .

```
In [6]: # Daftar isi dari setiap folder yang direpresentasikan dengan edge.
edge = [("Home", "Projects"),
        ("Home", "Music"),
        ("Home", "Downloads"),
        ("Projects", "App1.py"),
        ("Projects", "Report.pdf"),
        ("Music", "Song1.mp3"),
        ("Downloads", "Installer.exe")]

# Tambahkan edge ke dalam graf data_rina
data_rina.add_edges_from(edge)

# Tampilkan graf data_rina setelah penambahan edge
show_graph(data_rina, pos=pos)
```



Contoh output:

 Contoh output penambahan lokasi

d. (15 poin) Buat sebuah program yang memanfaatkan tree `data_rina` untuk mencetak setiap folder beserta daftar isi dari folder tersebut.

```
In [9]: print("Daftar Folder beserta isinya di komputer Rina")

# Iterasi untuk setiap node (folder/file) yang ada di graf `data_rina`
for node in data_rina.nodes():
    # Ambil semua node yang terhubung dengan node saat ini
    isi_folder = list(data_rina.neighbors(node))
```

```
# Jika folder memiliki isi, tampilkan daftarnya
if isi_folder :
    # Tampilkan folder beserta daftar isinya
    print(f"{node} berisi: {isi_folder}")
```

Daftar Folder beserta isinya di komputer Rina
 Home berisi: ['Projects', 'Music', 'Downloads']
 Projects berisi: ['App1.py', 'Report.pdf']
 Music berisi: ['Song1.mp3']
 Downloads berisi: ['Installer.exe']

Contoh output :

```
Daftar Folder beserta isinya di komputer Rina
- Home berisi: ['Projects', 'Music', 'Downloads']
- Projects berisi: ['App1.py', 'Report.pdf']
- Music berisi: ['Song1.mp3']
- Downloads berisi: ['Installer.exe']
```

e. (10 poin) Tampilkan urutan **folder dan file** yang dilalui dengan node asal **Home** menggunakan algoritma DFS yang disediakan dari NetworkX.

```
In [11]: print(list(nx.dfs_edges(data_rina, source = "Home")))
```

```
[('Home', 'Projects'), ('Projects', 'App1.py'), ('Projects', 'Report.pdf'), ('Home', 'Music'), ('Music', 'Song1.mp3'), ('Home', 'Downloads'), ('Downloads', 'Installer.exe')]
```

Contoh output :

```
[('Home', 'Projects'), ('Projects', 'App1.py'), ('Projects', 'Report.pdf'), ('Home', 'Music'), ('Music', 'Song1.mp3'), ('Home', 'Downloads'), ('Downloads', 'Installer.exe')]
```

2. Implementasi DFS (Penelusuran konsep DFS) (50)

a. (30 poin) Buatlah sebuah fungsi `dfs_search(graph, start, goal)` yang akan mencari dan menampilkan jalur dari node awal (`start`) ke node tujuan (`goal`) menggunakan algoritma DFS dengan deskripsi berikut:

- Gunakan stack untuk menyimpan node yang akan ditelusuri.
- Setiap elemen pada stack menyimpan (`current_node`, `path`) yang terdiri dari node saat ini dan jalur yang ditempuh hingga node tersebut.
- Lakukan iterasi DFS dengan memproses node terakhir dari stack.
- Jika `current_node` sama dengan `goal` , kembalikan jalur (`path`) yang telah ditemukan.
- Jika node saat ini memiliki tetangga yang belum dikunjungi, tambahkan tetangga tersebut ke stack.

```
In [35]: def dfs_search(graph, start, goal):
# Inisialisasi stack dengan tuple (node saat ini, jalur yang ditempuh)
stack = [(start, [start])]
```

```

# Iterasi selama stack tidak kosong
while stack:
    # Ambil node terakhir dari stack bersama jalurnya, dengan cara melakukan
    current_node, path = stack.pop()

    # Jika node saat ini adalah node tujuan, kembalikan jalur yang ditemukan
    if current_node == goal:
        return path

    # Tambahkan semua node tetangga yang belum dikunjungi ke stack
    for neighbor in graph.neighbors(current_node):
        # Pastikan tetangga belum ada di path untuk menghindari loop
        if neighbor not in path:
            # Tambahkan tetangga ke stack dengan jalur baru
            stack.append((neighbor, path + [neighbor]))

# Jika tidak ada jalur yang ditemukan, kembalikan None
return None # Biarkan NONE!! (jangan dirubah)

```

b. (20 poin) Buatlah sebuah program yang dapat melakukan langkah-langkah berikut:

- Meminta input pengguna sebagai titik awal folder pada variabel `start_node` .
- Meminta input pengguna untuk file yang dicari pada variabel `end_node` .
- Gunakan fungsi `dfs_search()` untuk menemukan urutan folder yang harus dikunjungi.
- Jika terdapat jalur dari `start_node` ke `end_node` , tampilkan folder mana saja yang harus dikunjungi.

```

In [33]: def start_dfs():
    # Minta input dari pengguna untuk node awal dan node tujuan
    start_node = input("Masukan folder/file awal: ")
    end_node = input("Masukan folder/file tujuan: ")

    # Temukan jalur dari titik awal ke titik tujuan menggunakan DFS
    path = dfs_search(data_rina, start_node, end_node)

    # Tampilkan hasil penelusuran
    if path:
        print(f"Rute yang harus dilalui dari {start_node} ke {end_node}: {path}")
    else:
        print("ooi")

```

```
In [36]: start_dfs()
```

Rute yang harus dilalui dari Home ke Song1.mp3: ['Home', 'Music', 'Song1.mp3']

```
In [31]: start_dfs()
```

Rute yang harus dilalui dari Home ke Report.pdf: ['Home', 'Projects', 'Report.pdf']

Contoh output:

input	output
Masukkan folder/file awal: Home	Rute yang harus dilalui dari Home ke Song1.mp3: ['Home', 'Music', 'Song1.mp3']

input	output
Masukkan folder/file tujuan: Song1.mp3	
Masukkan folder/file awal: Home	
Masukkan folder/file tujuan: Report.pdf	Rute yang harus dilalui dari Home ke Report.pdf: ['Home', 'Projects', 'Report.pdf']